

Lista01_MQ101_CerveraCelia

Nome: CELIA CERVERA CALLACNÁ

Data: 01/10/2025

Respostas da Lista #01

```
# Exercício 0 Preparação - Versão do R
R.version.string
```

```
## [1] "R version 4.5.1 (2025-06-13 ucrt)"
```

```
# Reprodutibilidade global para esta lista (voce pode mudar, mas mantenha constante)
set.seed(202501)
```

EXERCICIO 1 - R COMO CALCULADORA

```
10 + 2
```

```
## [1] 12
```

```
(10 + 2) * 3
```

```
## [1] 36
```

```
((10 + 2) * 3 - 6) / 3
```

```
## [1] 10
```

R respeita a ordem das operações. Isso é importante nas políticas públicas, por exemplo, quando se calculam taxas ou índices.

EXERCICIO 2 - OBJETOS E NOMEAÇÃO

```
x <- 1:6 #o vetor x contém dados de 1 a 6
Name <- 1
name <- 0
Name + 1
```

```
## [1] 2
```

```
name + 1
```

```
## [1] 1
```

R distingue entre maiúsculas e minúsculas. Neste caso, “Name” contém 1 e “name” contém 0.

EXERCICIO 3 - Sorteio e reprodutibilidade

```
set.seed(123)
die <- 1:6
sample(die, size = 2, replace = TRUE)
```

```
## [1] 3 6
```

Com o uso do setseed, os valores finais serão sempre iguais em todas as tentativas.

EXERCICIO 4 - Primeira função

```
roll2 <- function(bones = 1:6) {
  dice <- sample(bones, size = 2, replace = TRUE)
  #Sorteia dois valores do vetor "bones" com reposição.
  sum(dice) #suma os dois valores de "dice"
}
roll2()
```

```
## [1] 5
```

```
roll2(1:20)
```

```
## [1] 29
```

```
#implementar soma3()
soma3 <- function(bones = 1:6) {
  dice3 <- sample(bones, size = 3, replace = TRUE)
  sum(dice3)
}
soma3()
```

```
## [1] 15
```

EXERCICIO 5- Ajuda e exemplos

```
?sample
example(sample)
```

```
##
## sample> x <- 1:12
##
## sample> # a random permutation
## sample> sample(x)
## [1] 6 9 10 5 3 8 1 4 12 11 7 2
##
## sample> # bootstrap resampling -- only if length(x) > 1 !
## sample> sample(x, replace = TRUE)
## [1] 8 10 7 10 9 3 4 1 11 7 5 12
##
## sample> # 100 Bernoulli trials
## sample> sample(c(0,1), 100, replace = TRUE)
## [1] 0 1 1 0 0 0 0 1 0 0 1 0 0 0 0 1 1 0 1 0 0 1 1 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 1
```

```
## [38] 0 1 1 1 1 0 1 1 1 0 0 1 0 1 1 0 1 1 0 0 1 0 0 0 1 1 0 1 1 0 0 0 1 1 1 1 0
## [75] 0 1 1 1 0 1 0 1 0 0 0 1 1 0 0 0 1 0 1 1 1 1 1 1 1 0
##
## sample> ## More careful bootstrapping -- Consider this when using sample()
## sample> ## programmatically (i.e., in your function or simulation)!
## sample>
## sample> # sample()'s surprise -- example
## sample> x <- 1:10
##
## sample>      sample(x[x > 8]) # length 2
## [1] 10 9
##
## sample>      sample(x[x > 9]) # oops -- length 10!
## [1] 1 6 3 9 8 10 7 4 5 2
##
## sample>      sample(x[x > 10]) # length 0
## integer(0)
##
## sample> ## safer version:
## sample> resample <- function(x, ...) x[sample.int(length(x), ...)]
##
## sample> resample(x[x > 8]) # length 2
## [1] 9 10
##
## sample> resample(x[x > 9]) # length 1
## [1] 10
##
## sample> resample(x[x > 10]) # length 0
## integer(0)
##
## sample> ## R 3.0.0 and later
## sample> sample.int(1e10, 12, replace = TRUE)
## [1] 5904358143 5436121414 2654363467 2552658742 6719859143 5472473677
## [7] 2460433694 6877840345 9322834964 6051385898 9803396619 2947943816
##
## sample> sample.int(1e10, 12) # not that there is much chance of duplicates
## [1] 6347329389 8227823181 6623137529 4951625133 2844601333 4163368872
## [7] 5399656648 5443979253 721840007 8023502076 7190606655 6771070490
```

Os argumentos da amostra são x (vetor a partir do qual os valores serão selecionados), size (número de dados a serem selecionados), replace (TRUE significa com reposição) e prob (vetor de probabilidade de pesos).

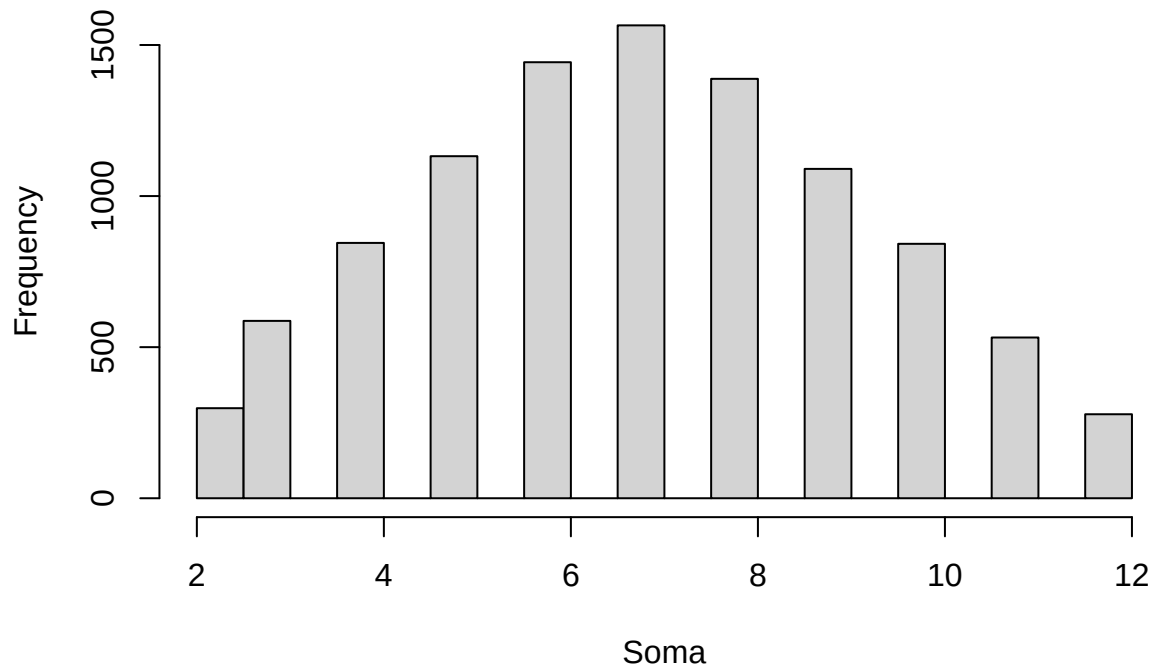
EXERCICIO 6 - SIMULAÇÃO E HISTOGRAMA

```
set.seed(42)
somas <- replicate(10000, roll2())
length(somas)
```

```
## [1] 10000
```

```
hist(somas, main = "Soma de dois dados (10.000 lançamentos)", xlab = "Soma")
```

Soma de dois dados (10.000 lançamentos)



```
mean(somas)
```

```
## [1] 6.9532
```

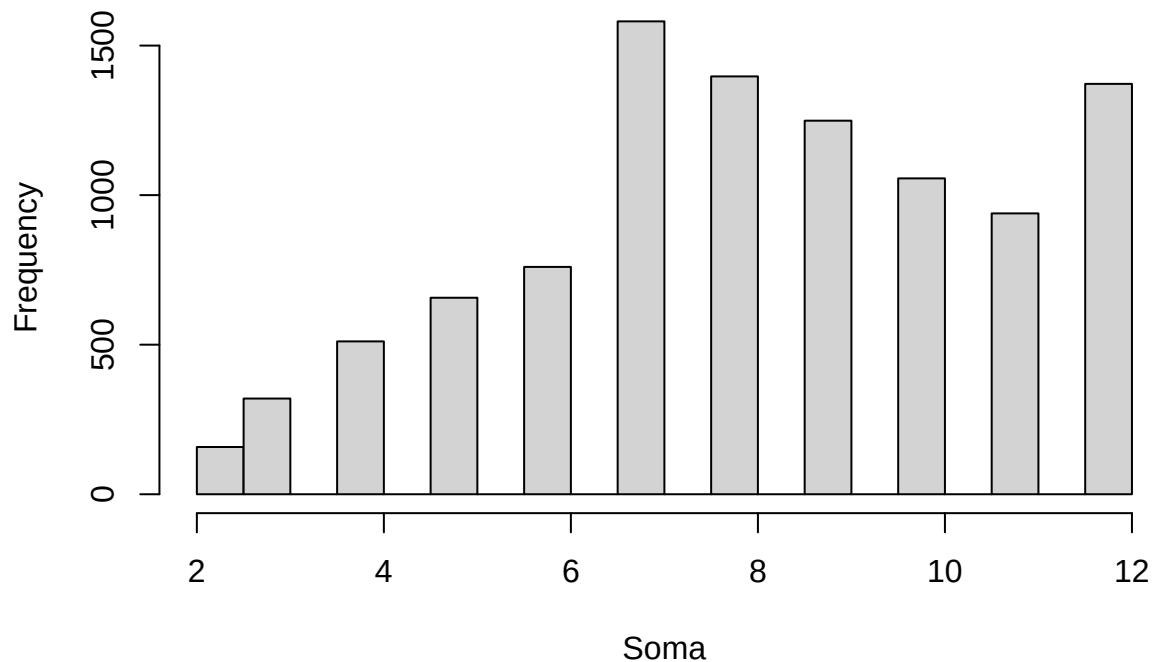
```
sd(somas)
```

```
## [1] 2.4329
```

O histograma mostra que o valor promedio de los datos é 6,9532 (média). Os dados apresentam um desvio padrão igual a 2,4329, o que indica a distribuição dos dados em torno da média.

```
# Extensão (opcional, sem pontos extras): dado viciado favorecendo o 6
prob_vies <- c(rep(1/8, 5), 3/8)
somas_vies <- replicate(10000, sum(sample(1:6, size = 2, replace = TRUE, prob = prob_vies)))
hist(somas_vies, main = "Dado viciado (6 favorecido)", xlab = "Soma")
```

Dado viciado (6 favorecido)



```
mean(somas_vies)
```

```
## [1] 8.2002
```

```
sd(somas_vies)
```

```
## [1] 2.605379
```

Ao atribuir um vetor de probabilidade, a média aumenta porque foi atribuída ao valor 6 uma maior probabilidade de ser escolhido.

EXERCICIO 7 - TIPOS BÁSICOS

```
dbl <- c(1.5, 2.0)           # numericos (double)
int <- c(1L, 2L)             # inteiros
chr <- c("saude", "educacao") # texto
lgl <- c(TRUE, FALSE)        # logico
```

```
str(list(dbl = dbl, int = int, chr = chr, lgl = lgl))
```

```
## List of 4
## $ dbl: num [1:2] 1.5 2
## $ int: int [1:2] 1 2
## $ chr: chr [1:2] "saude" "educacao"
## $ lgl: logi [1:2] TRUE FALSE
```

O comando `str` serve para saber o tipo de vetor que está sendo apresentado. No caso do vetor `dbl`, trata-se de números, para `int` trata-se de números inteiros, para `chr` trata-se de caracteres de texto e, no caso de `lgl`, trata-se de valores lógicos. Em um caso prático, isso pode ocorrer quando temos um banco de dados de algum município e desejamos conhecer a natureza de cada variável para poder tratá-la.

EXERCICIO 8 - DATAFRAME

```
faces <- c("ace", "two", "three", "four", "five", "six", "seven",
           "eight", "nine", "ten", "jack", "queen", "king")
suits <- c("spades", "hearts", "diamonds", "clubs")
deck <- data.frame(
  face = rep(faces, times = 4),
  suit = rep(suits, each = 13),
  value = rep(1:13, times = 4)
)

nrow(deck); ncol(deck) # serve para mostrar o numero de linhas e colunas.
```

```
## [1] 52
```

```
## [1] 3
```

```
head(deck, 10) #o comando serve para mostrar as primeiras 10 linhas.
```

```
##      face    suit value
## 1    ace spades     1
## 2   two spades     2
## 3 three spades     3
## 4   four spades     4
## 5   five spades     5
## 6   six spades     6
## 7 seven spades     7
## 8 eight spades     8
## 9   nine spades     9
## 10  ten spades    10
```

Simulamos uma mini base de dados municipal:

```
# Mini-base municipal (dados simulados)
set.seed(2025)
municipios <- paste0("Mun_", sprintf("%02d", 1:10))
dados_munic <- data.frame(
  municipio      = municipios,
  gasto_saude_pc = round(runif(10, 200, 1200), 2),
  taxa_evasao    = round(runif(10, 0.00, 0.20), 3),
  taxa_desemprego = round(rnorm(10, 0.12, 0.03), 3)
)

#runif - todos tienen la posibilidad de ser elegidos
#rnorm - dist normal media 0,12 y desvio padrão 0,03

head(dados_munic)
```

```
##      municipio gasto_saude_pc taxa_evasao taxa_desemprego
## 1      Mun_01          932.62      0.087      0.108
## 2      Mun_02          675.76      0.190      0.067
## 3      Mun_03          714.22      0.131      0.107
## 4      Mun_04          698.43      0.005      0.143
## 5      Mun_05          980.28      0.094      0.152
## 6      Mun_06          704.25      0.171      0.114
```

```
summary(dados_munic)
```

```
##      municipio      gasto_saude_pc      taxa_evasao      taxa_desemprego
## Length:10      Min.       : 327.9      Min.       :0.0050      Min.       :0.0670
## Class :character 1st Qu.: 699.9      1st Qu.:0.0730      1st Qu.:0.1072
## Mode  :character Median : 741.9      Median :0.0905      Median :0.1160
##              Mean  : 774.6      Mean  :0.0998      Mean  :0.1216
##              3rd Qu.: 910.6      3rd Qu.:0.1467      3rd Qu.:0.1405
##              Max.   :1098.4      Max.   :0.1900      Max.   :0.1930
```

EXERCICIO 9 - SELEÇÃO E FILTROS

```
deck[1, ]
```

```
##      face      suit value
## 1      ace spades      1
```

```
deck[c(1,3,5), c("face","suit")]
```

```
##      face      suit
## 1      ace spades
## 3 three spades
## 5 five spades
```

```
deck[-(1:48), ]
```

```
##      face      suit value
## 49      ten clubs      10
## 50      jack clubs      11
## 51      queen clubs      12
## 52      king clubs      13
```

```
subset_hearts <- deck[ deck$suit == "hearts", ]
nrow(subset_hearts)
```

```
## [1] 13
```

```
evaz_alta <- dados_munic[ dados_munic$taxa_evasao > 0.10, ]
evaz_alta
```

```
##      municipio gasto_saude_pc taxa_evasao taxa_desemprego
## 2      Mun_02          675.76         0.190         0.067
## 3      Mun_03          714.22         0.131         0.107
## 6      Mun_06          704.25         0.171         0.114
## 9      Mun_09          844.67         0.152         0.081
```

```
nrow(evaz_alta) #número de linhas
```

```
## [1] 4
```

Neste caso, foram filtrados os municípios que apresentam uma taxa de evasão superior a 0,10, considerando todas as outras colunas.

EXERCICIO 10 - MODIFICANDO VALORES E NA

```
# Modificando valores (ases = 14)
deck2 <- deck #cria um novo vector com os dados de deck
deck2$value[c(13, 26, 39, 52)] <- 14 #Todos os valores das colunas 13, 26, 39
#e 52 serão substituídos pelo valor 14.
head(deck2, 13) #mostra as primeiras 13 linhas
```

```
##      face  suit value
## 1      ace spades    1
## 2      two spades    2
## 3     three spades    3
## 4      four spades    4
## 5      five spades    5
## 6       six spades    6
## 7     seven spades    7
## 8     eight spades    8
## 9      nine spades    9
## 10     ten spades   10
## 11     jack spades   11
## 12    queen spades   12
## 13     king spades   14
```

```
# Valores ausentes
vals <- c(NA, 1:5)
mean(vals) # retorna NA
```

```
## [1] NA
```

```
mean(vals, na.rm = TRUE) # ignora NA
```

```
## [1] 3
```

```
dados_m2 <- dados_munic
dados_m2$taxa_evasao[3] <- NA
dados_m2$gasto_saude_pc[7] <- NA

head(dados_m2, 7)
```



```
##      municipio gasto_saude_pc taxa_evasao taxa_desemprego
## 1      Mun_01          932.62         0.087         0.108
## 2      Mun_02          675.76         0.190         0.067
## 3      Mun_03          714.22          NA         0.107
## 4      Mun_04          698.43         0.005         0.143
## 5      Mun_05          980.28         0.094         0.152
## 6      Mun_06          704.25         0.171         0.114
## 7      Mun_07           NA         0.073         0.118
```

```
mean(dados_m2$taxa_evasao)          # NA
```

```
## [1] NA
```

```
mean(dados_m2$taxa_evasao, na.rm=TRUE) # media sem NA
```

```
## [1] 0.09633333
```

EXERCICIO 11 OPCIONAL

```
setup <- function(deck_init) {
  DECK <- deck_init # c????pia interna (estado)

  DEAL <- function() {
    # Devolve a primeira carta e atualiza o baralho interno removendo-a.
    card <- DECK[1, , drop = FALSE]
    DECK <- DECK[-1, , drop = FALSE]
    return(card)
  }

  SHUFFLE <- function() {
    # Reembaralha o baralho interno
    idx <- sample(seq_len(nrow(deck_init)), size = nrow(deck_init))
    DECK <- deck_init[idx, , drop = FALSE]
    invisible(NULL)
  }

  list(deal = DEAL, shuffle = SHUFFLE)
}

cards <- setup(deck)
cards$deal()
```

```
##      face      suit value
## 1    ace spades        1
```

```
cards$deal()
```

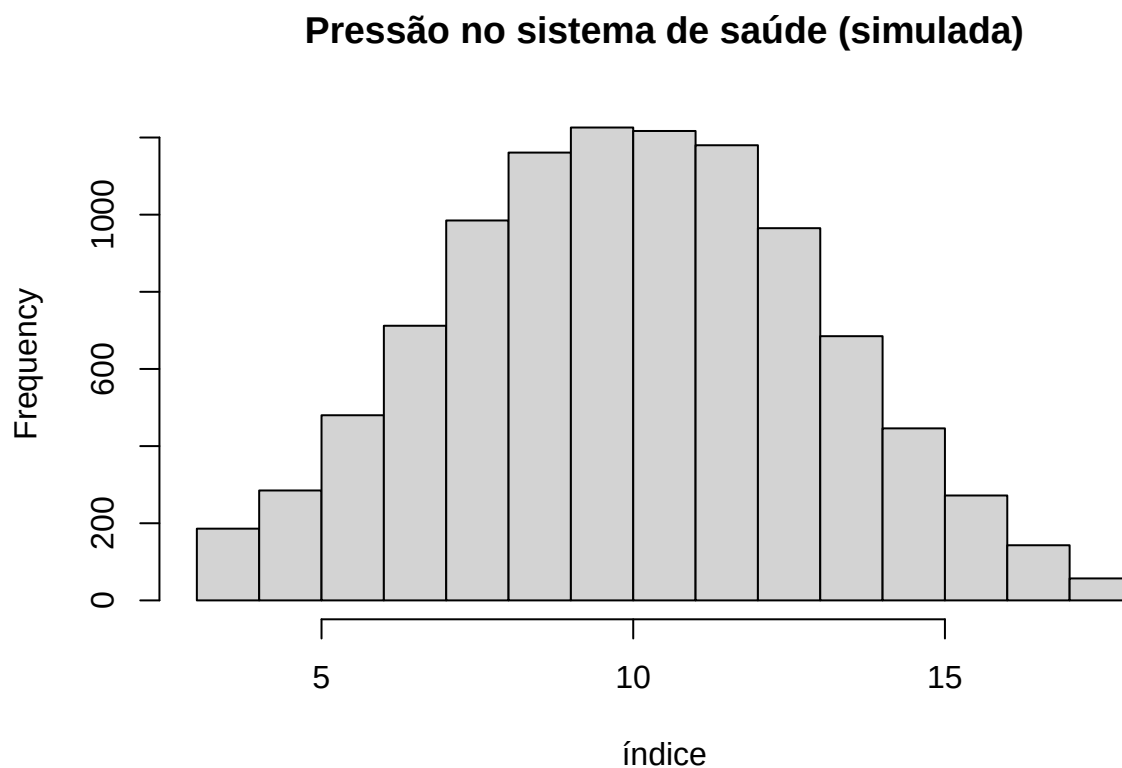
```
##      face      suit value
## 2    two spades        2
```

```
cards$shuffle()
cards$deal()
```

```
##   face   suit value
## 5 five spades    5
```

EXERCICIO 12 - MINI-PROJETO INTEGRADOR: SAÚDE

```
set.seed(123)
pressao_saude <- function() {
  demanda <- sample(1:6, 1, TRUE)
  equipe <- sample(1:6, 1, TRUE)
  insumos <- sample(1:6, 1, TRUE)
  demanda + equipe + insumos
}
prs <- replicate(10000, pressao_saude())
hist(prs, main = "Pressão no sistema de saúde (simulada)", xlab = "índice")
```



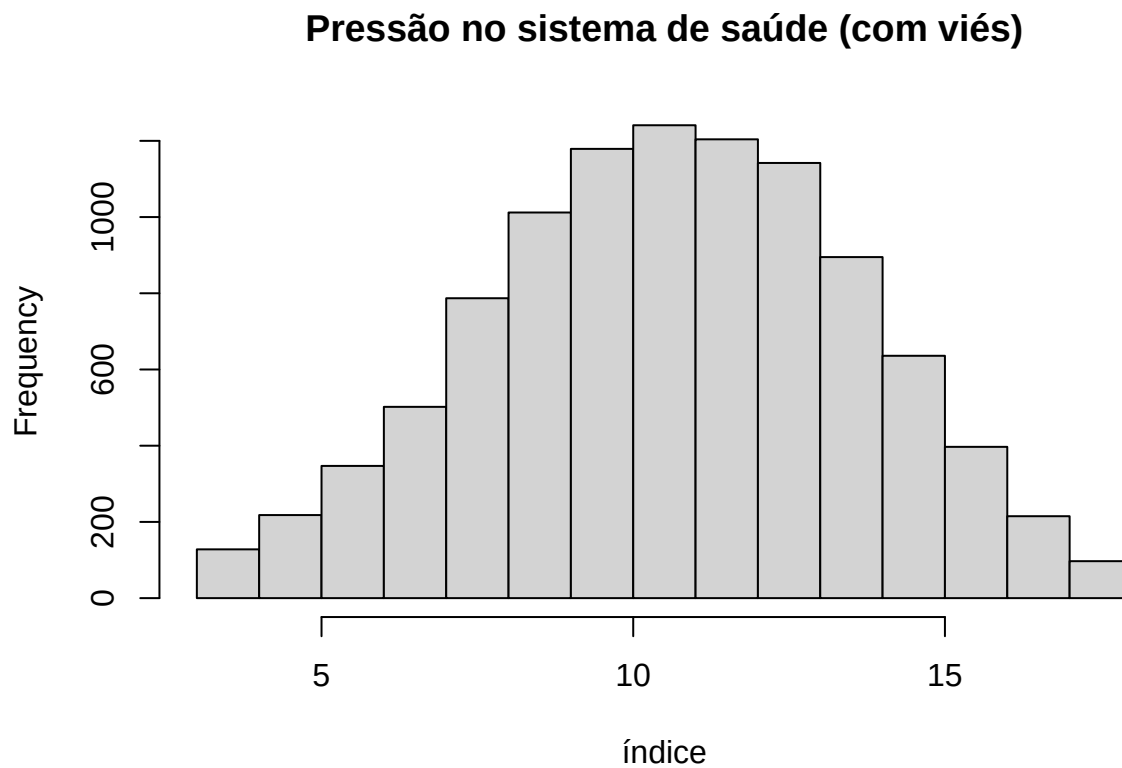
```
mean(prs)
```

```
## [1] 10.4759
```

```
sd(prs)
```

```
## [1] 2.97498
```

```
# Viés em demanda (favorece 6)
prob_demanda <- c(rep(1/8, 5), 3/8)
pressao_vies <- function() {
  demanda <- sample(1:6, 1, TRUE, prob = prob_demanda)
  equipe <- sample(1:6, 1, TRUE)
  insumos <- sample(1:6, 1, TRUE)
  demanda + equipe + insumos
}
prs_bias <- replicate(10000, pressao_vies())
hist(prs_bias, main = "Pressão no sistema de saúde (com viés)", xlab = "índice")
```



```
mean(prs_bias)
```

```
## [1] 11.1136
```

```
mean(prs)
```

```
## [1] 10.4759
```

A média dos dados de “prs_bias” (11,0612) é maior do que a média dos dados de ‘prs’ (10,4759), porque em “prs_bias” foi atribuído um vetor de probabilidade para a “demanda” que favorece o valor 6. Portanto, os valores de “prs_bias” serão maiores do que os de “prs”.

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8  LC_CTYPE=Portuguese_Brazil.utf8
## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## loaded via a namespace (and not attached):
## [1] compiler_4.5.1    fastmap_1.2.0     cli_3.6.5         tools_4.5.1
## [5] htmltools_0.5.8.1 rstudioapi_0.17.1 yaml_2.3.10       rmarkdown_2.29
## [9] knitr_1.50        xfun_0.52         digest_0.6.37     rlang_1.1.6
## [13] evaluate_1.0.4

““
```